Operations Manual — Madagascar Freshwater Fish Conservation Platform

A single-document reference for running, maintaining, and adding content to the Madagascar Freshwater Fish Conservation Platform (MFFCP / "malagasyfishes"). Targets Aleksei (project owner) and any future maintainer or content contributor who needs to do day-to-day work without re-reading the planning docs.

This manual is **operational** — what to run, what to click, what to check. It does not duplicate planning specs. Where another doc is authoritative, it is linked rather than restated.

> ***Companion docs (read first if you're touching that surface):***
>
> - `CLAUDE.md` *— project overview, tier table, conservation-status sourcing rule, auth + i18n architecture notes.*
> - `docs/handover/i18n-corrections-workflow.md` *— the long-form translation-pipeline runbook.*
> - `docs/handover/auth-gate-11-foundation.md` *— the auth surface (NextAuth + Django Token combo).*
> - `OPERATIONS.md` *(repo root) — staging-environment runbook (SSH, deploys, backups, container restarts).*
> - `deploy/staging/README.md` *— first-time staging bootstrap.*

Table of contents

1. Overview
2. Local development setup
3. Server commands cheat-sheet
4. Content entry
5. Updating content
6. Translation workflow (i18n)
7. Cache invalidation
8. Auth and access tiers
9. Backup and restore
10. Deployment
11. Feature flags
12. Common operational scripts and runbooks
13. Troubleshooting
14. Glossary

---

1. Overview

What this is for

A one-paragraph orientation for a new maintainer who has never seen the project before. Read this before anything else; it makes the rest of the manual make sense.

MFFCP is a centralized open-source platform for Madagascar's ~79 endemic freshwater fish species. It combines:

- **Public species profiles** — taxonomy, IUCN status, distribution, ecology, husbandry guidance — open at Tier 1 (anonymous).
- **Restricted ex-situ coordination** — captive populations across zoos, aquariums, and CARES hobbyist breeders, with breeding recommendations and transfer tracking — Tier 3+ (Conservation Coordinator) only.
- **Field program tracking** — in-situ conservation projects with Darwin-Core-aligned occurrence records.
- **Cross-sector networking** — bridges zoos, researchers, and hobbyist breeders that historically did not share infrastructure.

The platform is designed to **complement, not replace** ZIMS, IUCN Red List, FishBase, and GBIF. It mirrors authoritative data (e.g. IUCN status) and publishes occurrence records back out via Darwin Core Archives.

User tiers

The system has **five access tiers**, codified in `accounts.User.access_tier` (integer, 1–5) and enforced by the DRF API + Django admin.

Tier	Role	Sees
1	Public (anonymous)	Species profiles, conservation status, generalized distribution
2	Registered Researcher	Tier 1 + occurrence data, published datasets, field-program summaries
3	Conservation Coordinator	Tier 2 + sensitive locations, breeding recommendations, transfer coordination, dashboard
4	Program Manager	Tier 3 + population genetics, studbook-level data, full institutional inventory
5	Administrator	Full system access, user management, data import/export, all admin write paths

A few mechanical points worth knowing:

- Anonymous (unauthenticated) traffic is treated as **Tier 1**.
- Tier 3 and Tier 4 users with an `institution` foreign key are **scoped to their own institution** in the admin (see `populations/admin.py:ExSituPopulationAdmin`) — they can read all populations but only edit rows for their own institution.
- Tier 5 (admin / superuser) bypasses institution scoping.
- Coordinate sensitivity for threatened species (CR/EN/VU) is enforced per-record, not per-tier-globally — the public map generalizes coordinates for `SpeciesLocality` rows where the species is `iucn_status in {CR,EN,VU}` or `location_sensitivity = override_sensitive` . Tier 3+ sees exact coordinates.

Where the platform deploys

- **Production frontend (Vercel):** `https://malagasyfishes.org`
- **Staging API (Hetzner VPS):** `https://api.malagasyfishes.org`
- **Local development:** Docker Compose stack on `localhost:{8000, 3000, 15432, 6379, 9000}`

The Next.js frontend is hosted on Vercel; the Django API + Postgres + Redis + Celery worker live on a single Hetzner VM behind Caddy. See `OPERATIONS.md` for the staging runbook in detail.

---

2. Local development setup

What this is for

Bringing up a working local environment from a fresh clone. After this section finishes, you can browse the admin, hit the API, and run management commands.

Prerequisites

- **Docker Desktop** (or Docker Engine + Compose v2) — runs Postgres+PostGIS, Django, Celery, Redis, MinIO.
- **pnpm** — the frontend uses pnpm. `npm install -g pnpm`.
- **Node.js 20+** — for the frontend.
- A `.env` file at the repo root with at minimum a `DJANGO_SECRET_KEY` and IUCN/DeepL credentials if you'll exercise those paths.

Bring up the backend stack

From the repo root:

```
docker compose up -d
```

This brings up six services defined in `docker-compose.yml`:

- `db` — PostGIS 16-3.4 on host port `15432`
- `redis` — Redis 7 on host port `6379`
- `web` — Django on host port `8000` (uses `runserver` in dev so static files autoload without `collectstatic`)
- `worker` — Celery worker (consumes the Redis queue)
- `beat` — Celery beat (periodic-task scheduler; used for `iucn_sync` weekly)
- `minio` — S3-compatible object store on `9000` (console at `9001`, user/pass `minioadmin` / `minioadmin`)

The root `.env` file is mounted into the `web`, `worker`, and `beat` containers via `env_file: .env` in `docker-compose.yml`. Copy `.env.example` to `.env` if you don't have one yet:

```
cp .env.example .env
```

Apply migrations and seed reference data

```
docker compose exec web python manage.py migrate
docker compose exec web python manage.py seed_all
```

`seed_all` walks `/data/reference/` (HydroBASINS + WDPA shapefiles) and `/data/seed/*.csv` (species + localities). All three sub-steps are idempotent — re-running is safe.

Create an admin user

```
docker compose exec web python manage.py createsuperuser
```

Email + password. Superusers are auto-promoted to `access_tier=5`.

Bring up the frontend

```
cd frontend
pnpm install
pnpm dev
```

The frontend reads `frontend/.env.local`; copy from `frontend/.env.example` and fill in the secrets it expects (`NEXT_PUBLIC_API_URL`, `NEXTAUTH_SECRET`, `NEXT_REVALIDATE_SECRET`, etc.).

Common URLs

URL	What it serves
`http://localhost:8000/admin/`	Django admin (login with the superuser above)
`http://localhost:8000/api/v1/`	DRF browsable API root
`http://localhost:8000/api/v1/schema/`	OpenAPI schema (drf-spectacular)
`http://localhost:8000/api/v1/schema/swagger/`	Swagger UI
`http://localhost:3000/`	Next.js frontend (English)
`http://localhost:3000/fr/`	Next.js frontend (French) — flag-gated
`http://localhost:3000/dashboard/coordinator/`	Tier-3+ coordinator dashboard
`localhost:15432`	Postgres (user `postgres`, pw `postgres`, db `mffcp`)
`http://localhost:9001/`	MinIO console

---

3. Server commands cheat-sheet

What this is for

A quick reference for every Django management command worth knowing. All commands run inside the `web` container; locally that means `docker compose exec web python manage.py <cmd>`. On staging it means `docker compose exec web python manage.py <cmd>` from `~deploy/madagascarfish/deploy/staging/`.

> *Two things that look like management commands but aren't:*
>
> - ***`iucn_sync`*** *is a Celery task (`backend/integration/tasks.py`), scheduled weekly via `django-celery-beat`. To kick it manually, open a Django shell and call `from integration.tasks import iucn_sync; iucn_sync()`. There is no `python manage.py iucn_sync`.*
> - ***`import_iucn_taxonomy`*** *does not exist as a separate command. The `seed_species` command has an `--iucn-lookup` flag that populates `iucn_taxon_id` for matching binomials by hitting the IUCN scientific- name endpoint. Once `iucn_taxon_id` is set, the weekly `iucn_sync` task pulls assessments.*

Migrations and admin users

`migrate`

Apply database migrations. Run after every pull that touches `migrations/`. Idempotent (Django tracks applied migrations).

```
docker compose exec web python manage.py migrate
```

`createsuperuser`

Create a Tier-5 admin. Interactive — prompts for email, name, password. Not idempotent (will fail if the email already exists).

```
docker compose exec web python manage.py createsuperuser
```

`collectstatic`

Copy static assets into `STATIC_ROOT` (`/app/staticfiles`). Required in **production** because the prod stack uses Gunicorn + Caddy (no `runserver` to auto-serve admin CSS). Not needed in dev — `runserver` handles it. Idempotent.

```
docker compose exec web python manage.py collectstatic --noinput
```

Test-helper commands (dev/CI only)

These commands refuse to run unless `ALLOW_TEST_HELPERS=true` in the environment. They print credentials/tokens to stdout, which is fine for CI but a leak vector in production.

`seed_test_users`

Idempotent. Creates three deterministic users for the Playwright e2e:

- `researcher-e2e@example.com` (Tier 2)
- `coordinator-e2e@example.com` (Tier 3, scoped to "E2E Test Zoo")
- `admin-e2e@example.com` (Tier 5, staff)

All share the password `e2e-test-password-1234`.

```
docker compose exec web python manage.py seed_test_users
```

`get_verification_token --email <addr>`

Print the email-verification token for a not-yet-verified user. Used by the Gate 11 Playwright e2e to bypass the email vendor.

```
docker compose exec web python manage.py get_verification_token \
    --email researcher-e2e@example.com
```

Seed commands

All seed commands are idempotent — re-running them updates existing rows keyed on a natural key (scientific name, institution name, etc.) rather than duplicating.

`seed_all` *(recommended path)*

End-to-end seed of a fresh environment. Runs `load_reference_layers`, `seed_species`, then `seed_localities` from `/data/`. Each can be skipped with `--skip-reference`, `--skip-species`, `--skip-localities`.

```
docker compose exec web python manage.py seed_all
```

`seed_species --csv <path>`

Idempotent species loader, keyed on `scientific_name`. Required CSV columns: `scientific_name, family, genus, endemic_status, taxonomic_status`. Optional `--iucn-lookup` populates `iucn_taxon_id` by hitting the IUCN name endpoint after the seed completes (strict binomial match, provisional taxa skipped).

```
docker compose exec web python manage.py seed_species \
    --csv /data/seed/madagascar_freshwater_fish_seed.csv
```

`--dry-run` validates without writing.

`seed_localities --csv <path>`

Idempotent locality loader. Key: `(scientific_name, latitude, longitude, locality_name)`. Required: `scientific_name, latitude, longitude, locality_name, locality_type, source_citation`. Records outside Madagascar's bounding box are flagged `needs_review` rather than rejected. Drainage basin assignment requires `Watershed` rows from `load_reference_layers`.

```
docker compose exec web python manage.py seed_localities \
    --csv /data/seed/madagascar_freshwater_fish_localities_seed.csv
```

`load_reference_layers`

Load HydroBASINS watersheds and/or WDPA protected-area shapefiles into PostGIS. Idempotent on `hybas_id` / `wdpa_id`. Requires GDAL in the container (already in the Dockerfile).

```
docker compose exec web python manage.py load_reference_layers \
    --watersheds /data/reference/hydrobasins_madagascar_lev06.shp \
    --protected-areas /data/reference/wdpa_madagascar.shp
```

`--simplify 0.001` (default ~100m tolerance) keeps geometries small; pass `0` to skip simplification.

`seed_populations --csv <path>`

Load `ExSituPopulation` rows (and their parent `Institution` rows) from CSV. Institutions deduplicated by name. See the docstring in `backend/populations/management/commands/seed_populations.py` for the column shape.

```
docker compose exec web python manage.py seed_populations \
    --csv /data/seed/populations.csv
```

`seed_demo_coordination`

Seed plausible demo `CoordinatedProgram`, `Transfer`, `BreedingRecommendation`, and `BreedingEvent` rows for the coordinator dashboard. Every row is tagged with the marker `[DEMO_SEED]` at the start of a designated text field, so `--clear` can safely roll back the demo data without touching real operator-entered rows.

```
docker compose exec web python manage.py seed_demo_coordination
docker compose exec web python manage.py seed_demo_coordination --clear   # roll
back
docker compose exec web python manage.py seed_demo_coordination --dry-run # see what
would change
docker compose exec web python manage.py seed_demo_coordination --seed 42 # vary the
random seed
```

Idempotent: re-running without `--clear` is a no-op for existing demo rows.

`seed_husbandry`

Seed one fully-populated husbandry record on `Paretroplus menarambo`. Used to unblock frontend development against a realistically-shaped record. Idempotent — re-running updates the existing record.

```
docker compose exec web python manage.py seed_husbandry
```

`generate_map_layers --output-dir <path>`

Serialize watersheds and protected areas as static GeoJSON for the public map (avoids hitting the GIS backend on every page load).

```
docker compose exec web python manage.py generate_map_layers \
    --output-dir /app/staticfiles/map-layers/
```

`shoal_priority_report --csv <path>`

Read-only diff between a SHOAL priority CSV and the species registry. Makes zero writes — produces a three-section worklist (in-CSV-not-flagged, flagged-not-in-CSV, in-CSV-not-in-registry) for manual admin review.

```
docker compose exec web python manage.py shoal_priority_report \
    --csv /data/external/shoal_priority_2026.csv
```

i18n commands

`translate_species --locale <fr|de|es> [filters]`

Machine-translate Species, Taxon, and SpeciesHusbandry translatable fields via DeepL. Idempotent — skips fields whose target column already has content. Reads `DEEPL_API_KEY` from the environment (free-tier

keys end in `:fx`).

```
# Standard run — translate every empty French cell across all species.
docker compose exec web python manage.py translate_species --locale fr

# Restrict to one family (the @conservation-writer batched flow).
docker compose exec web python manage.py translate_species \
    --locale fr --family Bedotiidae

# Restrict to specific species ids.
docker compose exec web python manage.py translate_species \
    --locale fr --species 42 43 99

# Force-overwrite existing translations (use sparingly).
docker compose exec web python manage.py translate_species --locale fr --force

# Re-translate only rows whose English source changed and were auto-demoted.
docker compose exec web python manage.py translate_species \
    --locale fr --retranslate-stale

# Plan without calling DeepL — prints the job list.
docker compose exec web python manage.py translate_species --locale fr --dry-run

# Skip Taxon common_family_name translation.
docker compose exec web python manage.py translate_species --locale fr --no-taxon

# Skip SpeciesHusbandry's seven narrative/notes fields per species.
docker compose exec web python manage.py translate_species --locale fr --no-
husbandry
```

After this command runs, every translated field has a corresponding `TranslationStatus(status='mt_draft')` row. Advance through the pipeline in the admin — see [§6 Translation workflow](#) and `docs/handover/i18n-corrections-workflow.md` .

`makemessages -l <locale>`

Standard Django command — extract translation strings from Python/HTML sources into `locale/<locale>/LC_MESSAGES/django.po` . Used for **server- side gettext catalogs only** (e.g. error messages from `gettext_lazy(...)`). The frontend's UI strings live in `frontend/messages/<locale>.json` and are managed separately.

```
docker compose exec web python manage.py makemessages -l fr
docker compose exec web python manage.py makemessages -l de
docker compose exec web python manage.py makemessages -l es
```

`compilemessages`

Compile `.po` → `.mo` so Django can serve them at runtime. The Dockerfile runs this at build time (`RUN python manage.py compilemessages || true`), so prod images already have current `.mo` files baked in. Run locally if you've just edited `.po` files and want to test without rebuilding the container.

```
docker compose exec web python manage.py compilemessages
```

> ***Note:*** *`.mo` files are **not committed** (build artifact). `.po` files are the source of truth.*

Standard Django introspection

```
# Open a Django shell with all models pre-loaded.
docker compose exec web python manage.py shell

# Show every URL the project exposes.
docker compose exec web python manage.py show_urls   # if django-extensions
installed

# Dump a fixture (rare — pg_dump is preferred for full snapshots).
docker compose exec web python manage.py dumpdata species.Species \
    --indent 2 > species.json
```

---

4. Content entry

What this is for

Step-by-step instructions for adding each kind of data through the Django admin (`http://localhost:8000/admin/` or `https://api.malagasyfishes.org/admin/` on staging). Everything below assumes you're logged in as a Tier-5 superuser unless otherwise noted.

For bulk imports, prefer the seed commands in §3 over the admin — they're idempotent and CSV-driven.

Species

Admin: **Species → Species → Add Species**.

Required fields:

- `scientific_name` — e.g. `Paretroplus menarambo`. Treated as the natural key.
- `family` — e.g. `Cichlidae`. String column (a `Genus` FK is added on save).
- `genus` — string column. The model's `save()` auto-creates a `Genus` row if it doesn't exist and links `genus_fk`. You can also choose an existing `Genus` directly via the autocomplete on `genus_fk`.
- `taxonomic_status` — `described` / `undescribed_morphospecies` / `species_complex` / `uncertain` (default `described`).
- `endemic_status` — `endemic` / `native` / `introduced` (default `endemic`).

Conservation status:

- `iucn_status` is **read-only on the admin form**. It is a denormalized mirror of the most recently accepted `ConservationAssessment`. Do not reach for it directly. To change a species' IUCN category, create a `ConservationAssessment` row with `source=manual_expert` (see below).
- `cares_status` — editable; one of `CCR/CEN/CVU/CLC` or blank.
- `shoal_priority` — boolean. Set from the SHOAL priority list; flip via `shoal_priority_report` worklist.

- `location_sensitivity` — `auto` (default; CR/EN/VU → coordinates generalized) or `override_sensitive` (always generalized). **Superuser-only**.

Optional fields worth knowing:

- `silhouette_svg` — paste an inline `<svg>...</svg>` block, or use the "Upload .svg file" widget on the form to load one from disk. Width/height are stripped on save; a `viewBox` is synthesized if missing. Renders ~300px on the public profile.
- `iucn_taxon_id` — the IUCN Red List taxon SIS id (integer). Required for the weekly `iucn_sync` task to pull assessments for this species.
- `fishbase_id`, `gbif_taxon_key` — optional cross-references.

After save, the Species admin fires the **Revalidate public pages** action automatically (the species profile and listing pages rebuild on Vercel within a few seconds — see §7 Cache invalidation).

CommonName

Admin: inline on the Species change page.

One row per (species, language) pair. Fields:

- `name` — the common name itself (e.g. `Menarambo cichlid`).
- `language` — IETF language tag (`en`, `fr`, `de`, `es`, `mg`, etc.).
- `is_preferred` — only one preferred name per language per species.

Add as many as needed — multiple English common names, French, Malagasy, all coexist.

ConservationAssessment

Admin: **Species → Conservation assessments → Add**.

This is the **only correct path** for changing a species' IUCN category by hand. Editing `Species.iucn_status` directly is disabled by design (see `CLAUDE.md` "Conservation status sourcing").

Tier requirement: Tier 3 (Conservation Coordinator) or higher to add or edit. Enforced in `species/admin.py:ConservationAssessmentAdmin`.

Required fields when `source = manual_expert`:

- `species` — the species this assessment applies to.
- `category` — `CR/EN/VU/NT/LC/DD/EX/EW/NE`.
- `assessor` — your name + role (e.g. "A. Saunders, MFFCP coordinator").
- `assessment_date` — date of the assessment.
- `notes` — free text rationale.
- `reason` — separate free-text field on the admin form (not a model column). **Required when source is manual_expert.** Recorded in the audit log as the override justification.

After save, the `iucn_sync` mirror policy picks up the new assessment and updates `Species.iucn_status` to match `category`. A conflict between a `manual_expert` row and an incoming `iucn_official` row triggers a `ConservationStatusConflict` — see the species admin's "Recent iucn_status changes" panel (visible Tier 3+) or the `audit/AuditEntry` log.

`source = iucn_official` rows are normally created by the weekly `iucn_sync` Celery task; you can also create them manually if you're backfilling from a published Red List PDF.

Taxon (genus / family hierarchy)

Admin: **Species → Taxa**. MPTT-managed tree.

Most projects don't need to touch this — `Species.save()` auto-creates `Genus` rows from the string column, and the `Taxon` tree is supplemental for hierarchy navigation. If you're correcting a misclassification, change the `family` and `genus` strings on the Species, save, and verify the `Genus` row links correctly.

Family-level common names (e.g. "rainbowfishes" for Bedotiidae) are authored on the family-rank `Taxon` row in the `common_family_name` field. This field is `django-modeltranslation`-registered — it has `_en`, `_fr`, `_de`, `_es` columns.

Husbandry record (SpeciesHusbandry)

Admin: **Husbandry & Breeding Guidance → Species husbandry records → Add**.

One per species (`OneToOneField`). Fieldsets mirror the authoring template: **Water / Tank / Diet / Behavior / Breeding / Difficulty / Sourcing / Narrative / Governance**.

Publish-time validation (enforced in `husbandry/admin.py:save_related`):

- `published=True` requires:
  - At least one `HusbandrySource` inline row.
  - `last_reviewed_by` set (the reviewer user).
  - `last_reviewed_at` set (the review date).

If any of those are missing on save, the publish flag silently flips back to `False` and the admin shows form-level errors. Save again with the gaps filled in.

Stale-review banner: when `last_reviewed_at` is older than 24 months, the change form shows a warning and the public page renders a "review pending" note. This does **not** auto-unpublish — the record stays live but flagged for re-review.

Difficulty fields: seven separate factors (`difficulty_adult_size`, `difficulty_space_demand`, etc.). There is **no aggregate difficulty column** — the page surfaces factors, not a verdict. Locked design decision.

HusbandrySource

Admin: inline on the Species husbandry change page.

Fields: `order`, `label`, `url`. Order them sensibly — peer-reviewed literature first, breeder reports second, etc. At least one is required to publish.

Institution

Admin: **Populations → Institutions → Add**.

Fields:

- `name` — natural key. Used for dedup in `seed_populations`.
- `institution_type` — `zoo`, `aquarium`, `research_org`, `hobbyist_program` (CARES, Citizen Conservation), `hobbyist_keeper` (individual), `ngo`, `government`.
- `country`, `city`.
- `zims_member`, `eaza_member`, `aza_member` — booleans for membership badges.
- `species360_id` — ZIMS member id, if known.

- `contact_email` — visible only at Tier 3+.

Tier-3+ users with their `User.institution` set to a given `Institution` can write to `ExSituPopulation` and related rows scoped to that institution. This is the mechanism that lets a CARES coordinator update their own population without seeing every other zoo's data.

ExSituPopulation

Admin: **Populations → Ex situ populations → Add**.

One row per (species, institution). Fields cover counts, breeding status, date established, last census, notes.

Tier-3/4 institution scoping (from `populations/admin.py:ExSituPopulationAdmin`):

- Tier 3 or 4 users with `institution_id` set see only **their own** institution's populations on the change list.
- Cannot create/edit a row for any other institution — `save_model` raises `PermissionDenied` if `obj.institution_id != request.user.institution_id`.
- Tier 5 (admin) bypasses scoping.

`HoldingRecord` rows are inlined — these are point-in-time count snapshots (e.g. monthly census).

CoordinatedProgram

Admin: **Populations → Coordinated programs → Add**.

Above- `ExSituPopulation` layer. Captures the "who runs this": AZA SSP, EAZA EEP, CARES priority listing, or independent regional program.

Fields:

- `species` — one species per program.
- `program_type` — `ssp` / `eep` / `cares` / `independent` / `other`.
- `status` — `planning` / `active` / etc.
- `coordinating_institution` — holds the studbook.
- `studbook_keeper` — User FK.
- `enrolled_institutions` — many-to-many of partner zoos / aquariums / keepers (autocomplete widget).
- `target_population_size` — integer.
- `plan_summary`, `plan_document_url`, `start_date`, `next_review_date`.

This is the surface that drives the **Tier-3 coordinator dashboard** at `/dashboard/coordinator/`. See `docs/EAZA_EEP_ENTRY_GUIDE.md` for an end-to-end example walkthrough of adding a real program.

BreedingRecommendation (the "events" referenced as `BreedingRecommendationEvent`)

Admin: **Populations → Breeding recommendations → Add**.

> ***Naming note:*** *the model is `BreedingRecommendation`, not `BreedingRecommendationEvent`. Tracked lifecycle states (`open`, `in_progress`, `completed`, `superseded`, `cancelled` — default `open`) and resolution dates are fields on this single model, not a separate event table.*

Fields:

- `species`, `recommendation_type`, `priority`, `rationale`.

- `coordinated_program`, `source_population`, `target_institution`.
- `status`, `issued_date`, `due_date`, `resolved_date`, `outcome_notes`.
- `issued_by` (auto-set to current user on create), `resolved_by` (auto-set when status transitions to a terminal state).

Transfer (the "TransferActivity" exposed via API)

Admin: **Populations → Transfers → Add**.

> ***Naming note:*** *the model is `Transfer`. The `TransferActivityView` API endpoint at `/api/v1/coordinator-dashboard/transfer-activity/` is a* ***read-only view over `Transfer` rows*** *for the coordinator dashboard's recent-activity panel. Don't look for a `TransferActivity` model — there isn't one.*

Fields: `species`, `source_institution`, `destination_institution`, `coordinated_program`, `status` (lifecycle), `proposed_date`, `planned_date`, `actual_date`, counts (M/F/U), `cites_reference`, notes.

Set `actual_date` when `status='completed'`; leave blank until then.

BreedingEvent (the "ReproductiveActivity" exposed via API)

Admin: **Populations → Breeding events → Add**. (A `BreedingEventInline` class exists in `populations/admin.py` but is not currently wired onto `ExSituPopulationAdmin` — only `HoldingRecordInline` is. Add via the dedicated Breeding-events page.)

> ***Naming note:*** *the model is `BreedingEvent`. The `ReproductiveActivityView` API endpoint at `/api/v1/coordinator-dashboard/reproductive-activity/` is a* ***read-only view over `BreedingEvent` rows****.*

Fields: `population`, `event_type`, `event_date`, signed count deltas (`count_delta_male`, etc.), notes, reporter (auto-set).

Convention: a mortality of three males is `count_delta_male = -3`; a spawning that recruited five fry is `count_delta_unsexed = +5`. Leave blank when the event didn't change counts (e.g. recording a spawning before hatching).

Occurrence records (Darwin Core) — actually `SpeciesLocality`

Admin: **Species → Species localities → Add**.

> ***Naming note:*** *the project does not have a dedicated `OccurrenceRecord` model. Darwin-Core-aligned occurrence records live in the `SpeciesLocality` model in `backend/species/models.py`. Each row is a point observation with coordinates, citation, and provenance.*

Fields:

- `species`, `locality_name`, `locality_type`, `presence_status`, `water_body`, `water_body_type`.
- `location` — single PostGIS `PointField`. The admin renders a map widget (via `GISModelAdmin`) for direct point placement; the `seed_localities` management command is the path that takes `latitude` + `longitude` as separate CSV columns.
- `coordinate_precision`, `is_sensitive` (superuser-only).
- `drainage_basin` — editable autocomplete FK to `Watershed`. `drainage_basin_name` is the readonly cached display string. Spatial-join auto-assignment runs in `seed_localities` (not in the

admin save path); set the FK manually when entering localities through the admin.
- `year_collected`, `collector`, `source_citation`.
- `needs_review` — auto-set when coordinates fall outside Madagascar's bounding box, or longitude > 50.6° (offshore). Records flagged this way do not render on the public map until cleared.

Tier 1–2 sees coordinates **generalized** (rounded to ~10km) for species with `iucn_status in {CR,EN,VU}` or `location_sensitivity = override_sensitive`. **Tier 3+** sees exact coordinates. Tier 2 also sees generalized for sensitive species — the cut is at Tier 3, not Tier 2.

FieldProgram

Admin: **Fieldwork → Field programs → Add**.

Lightweight model — `name`, `lead_institution`, `status`, `region`, `start_date`, `description`, `focal_species` (M2M), `partner_institutions` (M2M).

User (creating accounts and assigning tiers)

Two paths:

A) Via `createsuperuser` (Tier 5 only)

```
docker compose exec web python manage.py createsuperuser
```

Creates an active, staff, superuser, `access_tier=5` user.

B) Via the admin (any tier)

Admin: **Accounts → Users → Add user**.

1. Email + name + password (twice).
2. Set `access_tier` (1–5). Defaults to 2 (Researcher).
3. Set `institution` (FK) if Tier 3 or 4 — controls the institution scoping in the populations admin.
4. Set `is_active` and `is_staff` if the user should be able to log into the admin.

Privilege escalation guard: `access_tier`, `is_active`, `is_staff`, and `is_superuser` are **read-only for non-superusers** (see `accounts/admin.py:UserAdmin.get_readonly_fields`). Only a Tier-5/superuser can promote a user.

`User.locale` — preferred language (L4 S7)

Each user has a `locale` field (`en` / `fr` / `de` / `es`, default `en`). Two effects:

- **Transactional emails** (verify-email, future password-reset etc.) render in `User.locale` via `backend/i18n/email.py::send_translated_email`.
- **Default UI locale on first visit** for logged-in users (read by the frontend through `getServerTier()` / NextAuth session).

Auto-set at signup from the path-prefix locale (a user signing up at `/fr/signup` gets `locale='fr'`). Self-serve update available on `/account` via the locale picker (`PATCH /api/v1/auth/me/locale/`). Admins can also set it directly on the user-edit page.

Bulk content via fixtures and seeds

For lots of species or localities, use the seed commands in §3 rather than the admin. They're idempotent, validate the CSV up-front, and print a per-row outcome summary.

---

5. Updating content

What this is for

How to change content that already exists, without breaking the denormalization mirrors, the i18n review pipeline, or the audit trail.

Changing a species' IUCN status (the right way)

Do not edit `Species.iucn_status` directly — the admin makes the field read-only for exactly this reason. The status is a denormalized mirror of the most-recent-accepted `ConservationAssessment`.

Two correct paths:

A) The IUCN sync handles it automatically.

If the species has an `iucn_taxon_id` and the IUCN Red List publishes a new assessment, the weekly `iucn_sync` Celery task picks it up, creates a new `ConservationAssessment(source='iucn_official', review_status='accepted')` row, and updates `Species.iucn_status` to match (gated on the `ALLOW_IUCN_STATUS_OVERWRITE` setting, default True).

B) You author a manual override.

When a recent expert assessment supersedes the published Red List category, or when the species has no IUCN listing yet:

1. Admin → **Species → Conservation assessments → Add**.
2. Set `source = manual_expert`, fill required fields including the `reason` text box.
3. Save. The mirror picks up the change and updates `Species.iucn_status`.

If a manual override and an incoming IUCN row disagree, the IUCN row lands as `pending_review` and a `ConservationStatusConflict` is created. Adjudicate via the species admin's "Recent iucn_status changes" panel.

The full policy is in `CLAUDE.md` § "Conservation status sourcing (mirror policy)".

Updating husbandry without losing review-stale signals

The husbandry record has two governance fields:

- `last_reviewed_by` — User who reviewed.
- `last_reviewed_at` — Date of review.

When you make a **substantive** edit (water parameters, tank size, diet notes, narrative), update both fields to record the new review. The 24-month stale-review timer resets.

When you make a **trivial** edit (typo in the narrative), leave the governance fields alone. The `review_is_stale` calculation reflects when the content was last vetted, not when it was last keyboard-touched.

If you publish a record (`published=True`) and the form throws errors, check the three preconditions: ≥1 source, reviewer set, review date set. The form silently flips `published` back to `False` when validation fails — this is by design (record stays draft until fixed).

Retiring an Institution

Don't delete it. `Institution` is a foreign key on `ExSituPopulation`, `Transfer.source_institution`, `Transfer.destination_institution`, `CoordinatedProgram.coordinating_institution`, `CoordinatedProgram.enrolled_institutions`, `BreedingRecommendation.target_institution`, `User.institution`, and `FieldProgram.lead_institution` / `partner_institutions`. Deleting will either cascade or block depending on the relation, and historical breeding-recommendation lineage gets lost either way.

The current model **does not have an explicit `is_active` field**. Until one ships, mark the institution as retired through naming and notes:

1. Edit the `name` to add a suffix: `"FooZoo (retired 2026-04-30)"` or `"FooZoo — closed"`.
2. If the institution had Tier 3/4 users, clear or reassign their `institution` FK so they don't keep editing under the retired name.
3. Document the retirement in the species/program records that referenced it (notes fields).

When a proper `is_active` field lands, this section will be updated.

Updating translations (FR/DE/ES)

Defer to the dedicated translation runbook — `docs/handover/i18n-corrections-workflow.md`. The two layers (UI catalog vs. species DB columns) have different correction paths; a fix in the wrong layer either won't propagate or won't survive the next MT pass.

The 60-second summary:

- **UI button labels, page headings, error microcopy** — edit `frontend/messages/<locale>.json`, run `pnpm i18n:check`, PR.
- **Species long-form prose** — Django admin → Translation pipeline → Translation statuses → click the row → edit "Target translation" → save. For substantive edits, also bulk-select the row and apply "Send back to mt_draft" so the change re-routes through review.

Editing the English source for a translated species

The `post_save` signal **auto-demotes** all locale rows back to `mt_draft` with a "needs re-review" note. The locale text isn't deleted — it's preserved in the column, but the public site (with `I18N_ENFORCE_REVIEW_GATE=true`) serves the English fallback until the row is re-approved.

To **retranslate** instead of just re-review:

```
docker compose exec web python manage.py translate_species \
    --locale fr --species 42 --force
```

---

6. Translation workflow (i18n)

What this is for

A short orientation pointing at the canonical runbook. The full operational flow (single-typo fixes, batched glossary changes, bulk corrections, re-translating from scratch, dumping/restoring local-to-prod) is in `docs/handover/i18n-corrections-workflow.md`. Use this section to get oriented, then click through.

Two layers, two correction paths

Layer	What it covers	Where it lives
1: UI	Page headings, button labels, form fields, errors, IUCN labels	`frontend/messages/{en,fr,de,es}.json`
2: DB	Species long-form prose, `Taxon.common_family_name`, husbandry notes	`*_en` / `*_fr` / `*_de` / `*_es` columns + `TranslationStatus`

Layer 1 is plain JSON files reviewed via PR. Layer 2 goes through a four-state pipeline tracked in `i18n.TranslationStatus`:

```
mt_draft → writer_reviewed → human_approved → published
```

The public site, gated by `I18N_ENFORCE_REVIEW_GATE=true`, serves only `human_approved` (or `published`) translations in the requested locale. Anything below that falls back to English with the `(English)` badge.

Current Bedotiidae state

As of this writing (2026-04-30), the **Bedotiidae batch sits at 29 rows in `writer_reviewed` state** — MT-translated by `translate_species`, then voice-reviewed by `@conservation-writer`, awaiting human approval through the L3 admin side-by-side review screen. To approve, go to **Translation pipeline → Translation statuses**, filter to `locale=fr`, `status=writer_reviewed`, content_type=Species, bulk-select, and apply **"Approve: writer_reviewed → human_approved"**.

Promotion stamps `human_approved_by` (the approving user) and `human_approved_at`. Once `I18N_ENFORCE_REVIEW_GATE=true` ships on staging, those rows go live in French immediately.

The MT pipeline command

```
docker compose exec web python manage.py translate_species --locale fr
```

Idempotent — only translates fields with no target-locale content. Reads `DEEPL_API_KEY` from the root `.env`. Free-tier keys end in `:fx`. Full flag list in §3.

Where the moving parts live

- `backend/i18n/management/commands/translate_species.py` — the MT command.
- `backend/i18n/signals.py` — auto-create / auto-invalidate logic.
- `backend/i18n/admin.py` — review surface (inline edit + actions).
- `backend/i18n/serializers.py` — `TranslationActualLocaleMixin` and the review gate.
- `frontend/scripts/translate-mt.mjs` — the catalog MT pipeline (Layer 1).
- `frontend/scripts/check-i18n-keys.mjs` — parity check.

For all operational detail, go to `docs/handover/i18n-corrections-workflow.md`.

---

7. Cache invalidation

What this is for

Public pages on the Vercel-hosted frontend are served via Next.js Incremental Static Regeneration (ISR) with a 3600-second revalidation window. When you edit content in admin, you don't want to wait an hour for it to

surface. This section covers the manual cache-bust path.

The "Revalidate public pages" admin action

Almost every content admin (`Species` , `Genus` , `Taxon` , `SpeciesLocality` , `Institution` , `ExSituPopulation` , `CoordinatedProgram` , `Transfer` , `BreedingRecommendation` , `BreedingEvent` , `ConservationAssessment` , `SiteMapAsset`) ships with an action labelled **"Revalidate public pages (clear Next.js cache)"**.

It also fires automatically inside `save_model` for Species, Genus, SiteMapAsset, Institution, ExSituPopulation, CoordinatedProgram, Transfer, BreedingRecommendation, and BreedingEvent — so a normal edit-and-save already revalidates without you clicking anything.

To trigger manually (e.g. you did a raw SQL update bypassing the admin):

1. Go to any model's admin list view that exposes the action.
2. Select one or more rows (the action ignores the queryset — selection doesn't matter, but Django requires at least one row checked).
3. **Action** dropdown → **"Revalidate public pages (clear Next.js cache)"** → **Go**.
4. The success message reports `"Revalidated N path(s)"` .

What gets revalidated

The canonical path list lives in `species/admin_revalidate.py:PUBLIC_PATHS` :

```
PUBLIC_PATHS = [
    "/",
    "/dashboard",
    "/species",
    "/species/[id]",
    "/map",
    "/about",
]
```

`/species/[id]` is a dynamic route — Next.js's `revalidatePath` handles the wildcard and rebuilds every cached species profile.

Required env vars

The Django side reads two settings (in `config/settings/base.py`):

- `NEXT_REVALIDATE_URL` — full URL to the Next.js revalidate webhook. Default: `http://localhost:3000/api/revalidate` . On staging: `https://malagasyfishes.org/api/revalidate` .
- `NEXT_REVALIDATE_SECRET` — shared secret. Sent in the `X-Revalidate-Secret` header. Must match the value Vercel reads for `REVALIDATE_SECRET` .
- `NEXT_REVALIDATE_TIMEOUT_SECONDS` — request timeout, default 10s.

If either of the first two is blank, the admin action shows `"Revalidate is not configured: set NEXT_REVALIDATE_URL and NEXT_REVALIDATE_SECRET in the environment."` and exits without firing.

The pair lives in:

- **Local dev:** repo root `.env` .

- **Staging Django:** `deploy/staging/.env` on the Hetzner VM.
- **Vercel (verifying side):** Vercel project → Settings → Environment Variables.

The canonical rotation procedure is in `OPERATIONS.md` §11.1.

---

8. Auth and access tiers

What this is for

How the five-tier access model is enforced at the API + admin level, plus how to upgrade a user's tier when somebody legitimately needs more access. For the wiring diagram (NextAuth + Django Token interplay, JWT cookie, session-token vs service-token resolution), see `docs/handover/auth-gate-11-foundation.md` and `CLAUDE.md` § "Auth (Gate 11)".

What each tier sees

The full table is in §1 Overview. Mechanically:

- **Tier 1 (anonymous):** The DRF API filters via per-model `for_tier()` querysets that drop `pending_review` ConservationAssessment rows, generalize coordinates for sensitive species, and hide `Institution.contact_email`. Public pages render against the same filtered API.
- **Tier 2 (researcher):** Same as Tier 1 plus access to additional research-data endpoints. Coordinate generalization for sensitive species still applies (the boundary for exact coordinates is Tier 3, not Tier 2).
- **Tier 3 (coordinator):** Exact coordinates, breeding recommendations, transfer coordination, the `/dashboard/coordinator/` panels. If `User.institution` is set, the populations admin scopes writes to that institution.
- **Tier 4 (program manager):** Adds full institutional inventory and studbook-level aggregates to Tier 3.
- **Tier 5 (admin):** Full system. Bypasses Tier 3/4 institution scoping. Only Tier 5 (or `is_superuser`) can edit `Species.location_sensitivity`, `SpeciesLocality.is_sensitive`, and the privilege fields on a User.

Tier-3+ scoping for institutions

The relevant logic is in `backend/populations/admin.py:ExSituPopulationAdmin`:

```
def _is_institution_scoped(self, request):
    user = request.user
    return (
        user.is_authenticated
        and hasattr(user, "access_tier")
        and 3 <= user.access_tier <= 4
        and user.institution_id is not None
    )
```

When that returns `True`:

- The change list filters to rows where `institution_id == request.user.institution_id`.
- `has_change_permission` and `has_delete_permission` reject objects from any other institution.

- `save_model` raises `PermissionDenied` if the form is bound to a different institution than the user's.

A Tier-3 coordinator at "FooZoo" can read every other zoo's populations through the API (no row-level read scoping), but can only **write** to FooZoo's. Tier 5 has no scoping.

Upgrading a user's tier

1. Log in to admin as a superuser (anything below Tier 5 cannot edit privilege fields — `access_tier`, `is_active`, `is_staff`, `is_superuser`).
2. **Accounts → Users → click the user**.
3. Edit `access_tier` (1–5).
4. If promoting to Tier 3 or 4, also set `institution` to the user's home institution. This is what activates the institution-scoping path.
5. Save.

The `User.access_tier` field carries a `CheckConstraint` rejecting values outside `[1, 5]`.

Logging out

NextAuth + Django Token = **dual-fire logout required**. The frontend's `signOut()` call clears the NextAuth cookie; a separate `POST /auth/logout/` is needed to delete the DRF Token row. Both happen automatically when a user clicks "Log out" in the UI; if you're scripting or hitting the API, fire both. See `CLAUDE.md` § "Auth (Gate 11)" for the full rule.

---

9. Backup and restore

What this is for

A repeatable Postgres snapshot/restore path. The full staging-environment runbook is in `OPERATIONS.md` §9. This section covers the same shape for local + cross-environment use cases.

Local snapshot

```
docker compose exec -T db pg_dump \
    -U postgres -d mffcp \
    --format=custom \
    --no-owner --no-privileges \
    > ~/mffcp-local-$(date +%Y%m%d).dump
```

Custom format = compressed, restorable with `pg_restore` (selective table restore possible).

Local restore

```
# Drop and recreate the DB first (destructive — confirm before running).
docker compose exec db psql -U postgres -c "DROP DATABASE IF EXISTS mffcp;"
docker compose exec db psql -U postgres -c "CREATE DATABASE mffcp;"
docker compose exec db psql -U postgres -d mffcp -c "CREATE EXTENSION postgis;"

# Restore the dump.
docker compose exec -T db pg_restore \
```

```
    -U postgres -d mffcp --no-owner --no-privileges \
    < ~/mffcp-local-20260430.dump

# Replay migrations to apply anything newer than the snapshot.
docker compose exec web python manage.py migrate
```

Selective dump (one or two tables)

Used by the i18n local-to-prod move (Path A in `i18n-corrections-workflow.md`):

```
docker compose exec -T db pg_dump \
    -U postgres -d mffcp \
    --data-only --column-inserts \
    --table=species_species \
    --table=i18n_translationstatus \
    > /tmp/mffcp-fr-snapshot.sql
```

`--data-only` skips schema; `--column-inserts` produces per-row INSERTs that are safe to inspect / cherry-pick.

Staging backup and restore

The staging VM has a `~/backups/` directory and a documented backup procedure in `OPERATIONS.md` §9. Use that runbook for staging — don't reinvent.

---

10. Deployment

What this is for

How code lands on staging. **Do not** confuse "staging" with "production" — at the time of writing, the Hetzner-hosted environment at `api.malagasyfishes.org` + the Vercel-hosted `malagasyfishes.org` is the only live deployment. There is no separately-provisioned production tier yet; the staging stack serves the public site.

For first-time bootstrap (provisioning the VM, installing Docker, wiring GitHub Actions), see `deploy/staging/README.md`.

Backend Dockerfile

`backend/Dockerfile` builds the Django container. Notable details:

- Base image: `python:3.12-slim`.
- System deps installed: `build-essential`, `gdal-bin`, `gettext`, `libgdal-dev`, `libgeos-dev`, `libproj-dev`, `postgresql-client`.
- `gettext` is required for `makemessages` / `compilemessages`.
- The build runs `python manage.py compilemessages || true` so committed `.po` files become runtime `.mo` files baked into the image. The `|| true` keeps the build green even before any catalog has been committed.
- Default CMD: `gunicorn config.wsgi:application --bind 0.0.0.0:8000 --workers 2`. This is what production uses. Local dev overrides via `docker-compose.yml` to `python manage.py`

`runserver 0.0.0.0:8000` — `runserver` autoloads admin/DRF static files without `collectstatic`.

There is **no separate deploy step inside the Dockerfile**. Deployment specifics (migrate, collectstatic, exec gunicorn) live in `deploy/staging/docker-compose.yml`'s `web` service command:

```
command: >
  sh -c "python manage.py migrate --noinput &&
         python manage.py collectstatic --noinput &&
         gunicorn config.wsgi:application --bind 0.0.0.0:8000 --workers 3 --access-logfile - --error-logfile -"
```

That runs every time the container starts.

Staging auto-deploy (workflow_run gate)

`.github/workflows/deploy-staging.yml` auto-deploys main to the Hetzner VPS after CI passes. The wiring is:

```
on:
  workflow_run:
    workflows: ["CI"]
    branches: [main]
    types: [completed]
  workflow_dispatch:

jobs:
  deploy:
    if: >
      github.event_name == 'workflow_dispatch' ||
      github.event.workflow_run.conclusion == 'success'
```

Two gotchas worth knowing (matches the user-memory note on `reference_deploy_staging`):

- **The `workflow_run` gate skips silently when CI is red.** If `main` merges and CI fails, the deploy workflow runs but does nothing — there is no failure log on the deploy workflow itself. Always check the CI workflow's status before suspecting the deploy.
- **`STAGING_SSH_HOST_KEY` must match `STAGING_SSH_HOST` exactly.** The workflow seeds `~/.ssh/known_hosts` from the secret; a mismatch causes every SSH attempt to fail with a "host key verification failed" error. Re-keying after a VM rebuild requires `ssh-keyscan -t ed25519 <host>` output back into the secret.

The deploy step itself SSHes into the VM, `git fetch && git reset --hard origin/main`, runs `docker compose up -d --build web worker`, and pings the API for a HTTP 200 on `/api/v1/schema/` (six retries, 10s apart).

Required GitHub repo secrets

Listed in the workflow comments:

- `STAGING_SSH_HOST` — IPv4 or hostname of the Hetzner VPS.
- `STAGING_SSH_USER` — deploy user (typically `deploy`).

- `STAGING_SSH_PRIVATE_KEY` — private key whose pubkey is in `~deploy/.ssh/authorized_keys`.
- `STAGING_SSH_HOST_KEY` — output of `ssh-keyscan -t ed25519 <host>`.

Other workflows

- `.github/workflows/ci.yml` — the main backend CI gate (lint, type check, pytest).
- `.github/workflows/frontend-ci.yml` — frontend type check / unit tests / i18n parity.
- `.github/workflows/frontend-e2e.yml` — Playwright e2e (frontend + backend).
- `.github/workflows/frontend-auth-e2e.yml` — Playwright e2e for the Gate 11 auth surface specifically.

Vercel (frontend production)

The Vercel project is connected to this repo's `frontend/` directory via the standard Vercel-GitHub integration. Pushes to `main` trigger a build

- deploy. Vercel reads its env vars from project settings — `NEXT_PUBLIC_*` flags, `NEXTAUTH_SECRET`, `REVALIDATE_SECRET`, etc. Per- preview-branch and per-production envs are set independently.

---

11. Feature flags

What this is for

Inventory of every flag that gates a major surface, where each lives, and what flipping it does.

Frontend flags (Vercel env vars, prefix `NEXT_PUBLIC_*`)

Flag	Default	Where set	Gates
`NEXT_PUBLIC_FEATURE_AUTH`	off	Vercel	The entire NextAuth surface — login/signup/verify/account pages, the `/account` redirect in `frontend/middleware.ts`, the auth links in the nav. With this off, the public site is anonymous-only.
`NEXT_PUBLIC_FEATURE_I18N`	off	Vercel	The header `<LocaleSwitcher />` widget. With this off the dropdown is hidden; locale-prefix routes (`/fr/...`) still work because next-intl middleware always runs.
`NEXT_PUBLIC_FEATURE_I18N_FR`	off	Vercel	Inclusion of French in the locale-switcher dropdown. The route `/fr/` is reachable regardless; this gate hides the *visible* link until French content is human-approved.
`NEXT_PUBLIC_FEATURE_I18N_DE`	off	Vercel	Same shape, German.
`NEXT_PUBLIC_FEATURE_I18N_ES`	off	Vercel	Same shape, Spanish.

Treat `NEXT_PUBLIC_*` flags as **public** — they're inlined into the client bundle. Don't put secrets there.

Backend flags (root `.env`)

Most are read in `config/settings/base.py`. Two exceptions noted in the table: `DEEPL_API_KEY` is read directly via `os.environ` in `translate_species`, and `COORDINATOR_API_TOKEN` is also consumed by the Next.js SSR layer (`frontend/lib/coordinatorDashboard.ts`) — set the same value in both Vercel and the Django env.

Flag	Default	Gates
`I18N_ENFORCE_REVIEW_GATE`	False	When True, only `human_approved` (or `published`) translation rows are served in their target locale. Below that, the API returns the English fallback. Stays False through L1–L2; flips True once enough rows are human-approved.
`ALLOW_IUCN_STATUS_OVERWRITE`	True	When True, `iucn_sync` mirrors accepted IUCN categories onto `Species.iucn_status`. Toggle off during a manual review window if you want the sync to log changes without touching the public badge.
`ALLOW_TEST_HELPERS`	False	Permits `seed_test_users` and `get_verification_token` to run. **Never set to True in prod** — these commands print credentials/tokens to stdout.
`TRUST_X_FORWARDED_FOR`	False	When True, login rate-limiting reads the client IP from `X-Forwarded-For` instead of `REMOTE_ADDR`. Set True only when Django sits behind a trusted reverse proxy (Caddy on staging). Spoofable if the WSGI port is reachable directly.
`COORDINATOR_API_TOKEN`	(empty)	Service token (consumed by both Django `settings.py` and Next.js SSR — set in Vercel env too). Lets server-side renderers (coordinator dashboard) hit Tier-3+ endpoints without a user session. Sent as `Authorization: Bearer <token>`. **Emergency-fallback only**; the session-token path is the default.
`NEXT_REVALIDATE_URL`	local	URL of the Next.js revalidate webhook. See §7.
`NEXT_REVALIDATE_SECRET`	(empty)	Shared secret for the revalidate webhook. Blank disables the admin "Revalidate public pages" action with a visible notice.
`IUCN_API_TOKEN`	(empty)	API key for the IUCN Red List v4 API.
`DEEPL_API_KEY`	(empty)	API key for DeepL. Read directly via `os.environ` in `translate_species` (not declared in `settings.py`). Required for any MT pipeline run. Free-tier keys end in `:fx`.

Where to flip flags

- **Frontend (Vercel):** Vercel project → Settings → Environment Variables. Choose the right environment (Production / Preview / Development). Redeploy after change.
- **Django (staging):** edit `~deploy/madagascarfish/deploy/staging/.env` on the Hetzner VM, then `docker compose up -d` from `deploy/staging/` to restart the affected containers.

- **Django (local):** edit the root `.env`, then `docker compose restart web worker beat`.

---

12. Common operational scripts and runbooks

What this is for

Pointers to detailed runbooks that already exist. Don't duplicate them — follow the link.

Topic	Runbook
Staging connect, deploys, restarts, log inspection, post-merge flow	`OPERATIONS.md`
First-time staging VM bootstrap	`deploy/staging/README.md`
i18n correction workflow (Layer 1 catalog + Layer 2 DB, glossary updates, prod move)	`docs/handover/i18n-corrections-workflow.md`
i18n flag-flip pre-flight + rollback (turning `..._FR=true` on prod safely)	`docs/operations/i18n-flag-flip-runbook.md`
Auth Gate 11 foundation (NextAuth + Django Token)	`docs/handover/auth-gate-11-foundation.md`
EAZA EEP coordination program entry — worked example	`docs/EAZA_EEP_ENTRY_GUIDE.md`
CARES reduced-scope plan	`docs/CARES_REDUCED_SCOPE_PLAN.md`
Conservation status mirror policy (the only correct way to change `iucn_status`)	`CLAUDE.md` § "Conservation status sourcing"
Active initiatives (Registry redesign etc.)	`docs/planning/<initiative>/README.md`

Frontend scripts

These live in `frontend/scripts/` and are wired into `frontend/package.json`. Run from `frontend/`:

```
pnpm i18n:check         # verify key parity across all four locale catalogs
pnpm i18n:lint-pockets  # CI guard — fail on hardcoded English in server-actions /
lib/husbandry
pnpm i18n:translate     # MT pipeline for the UI catalog (Layer 1)
pnpm bake-tiles       # offline bake of Esri map tiles for /map
pnpm gen:types        # regenerate lib/api-types.ts from /api/v1/schema/
pnpm gen:types:check  # verify lib/api-types.ts is in sync with the live schema
pnpm typecheck        # tsc --noEmit
pnpm lint             # next lint
pnpm test             # vitest run
pnpm e2e              # playwright test
```

---

13. Troubleshooting

What this is for

Symptoms you might hit and the fix that worked. Add to this list as new ones come up — the "what was happening" + "what fixed it" pairing is more useful in six months than the abstract reasoning.

Symptom: admin pages render unstyled (no CSS, no JS)

Cause: the `web` container is running `gunicorn` (production CMD) without `collectstatic` having been run, so there's no `/app/staticfiles/admin/css/*` for Caddy/Whitenoise to serve.

Fix: in dev, the `docker-compose.yml` overrides the CMD to `runserver`, which autoloads admin static files. If your local stack is unstyled, you've probably overridden `command:` somewhere — restore `runserver`. In staging, the `web` service runs `collectstatic` on every container start; if it's still unstyled, check that the `staticfiles:` named volume mounted into Caddy at `/srv/static`.

Symptom: admin login throws 429 / "too many attempts"

Cause: the per-IP login rate-limiter has cached an "over the limit" entry for the user's IP under a `cache` key.

Fix (dev):

```
docker compose exec web python manage.py shell -c \
    "from django.core.cache import cache; cache.clear()"
```

This clears Django's whole cache, including the rate-limit counters. In prod, prefer flushing only the affected key — find the prefix in the login view's rate-limit code and `cache.delete('<prefix>:<ip>')`.

If the user is **legitimately locked out** because they forgot their password, just wait for the window to elapse (default 15 minutes) or clear the cache.

Symptom: `translate_species` exits with `DEEPL_API_KEY is not set`

Cause: the env var isn't reaching the container. Check root `.env` has `DEEPL_API_KEY=...:fx` (free) or without `:fx` (paid), then restart:

```
docker compose restart web worker beat
```

Symptom: `translate_species` returns HTTP 403 from DeepL

Cause: the API key has been revoked, or it's a free-tier key being sent to the paid endpoint (or vice versa). The command auto-detects endpoint from the `:fx` suffix; if the suffix doesn't match the key, you get a 403.

Fix: verify the key in the DeepL account dashboard, paste the correct suffix into `.env`, restart containers.

Symptom: French URL renders English with `(English)` fallback badge

Cause: intentional. The serializer's review gate only emits target-locale content when `TranslationStatus.status == human_approved` (or `published`). One of:

- The translation hasn't been reviewed and approved yet.
- Someone edited the English source after approval — the post_save signal auto-demoted the locale row back to `mt_draft` with a "needs re-review" note.
- `I18N_ENFORCE_REVIEW_GATE=False` and the column is empty in the requested locale.

Fix: find the species in **Translation pipeline → Translation statuses**, filter to the locale + species, advance through the pipeline. See `i18n-corrections-workflow.md` for the full flow.

Symptom: admin shows "no rows" for Species, Translation statuses, etc.

Cause: the database is empty (fresh checkout, or just-restored from a schema-only dump). Migrations applied successfully, but no seed has run.

Fix:

```
docker compose exec web python manage.py seed_all
docker compose exec web python manage.py translate_species --locale fr
```

The first populates Species + localities; the second fills `*_fr` columns and creates the `TranslationStatus` rows the pipeline admin shows.

Symptom: editing a Species saves successfully but the public page still shows old content

Cause: Vercel's ISR cache hasn't been invalidated. The `save_model` hook in Species admin **does** fire `_post_revalidate()` automatically — but it can fail silently if `NEXT_REVALIDATE_URL` or `NEXT_REVALIDATE_SECRET` is misconfigured.

Fix:

1. Check that the success banner after save reads "Revalidated N path(s)" and not a configuration warning.
2. If it's the warning, fix the env vars (see §7) and restart the `web` container.
3. To force a manual revalidation, use the **"Revalidate public pages"** admin action.
4. As a last resort, redeploy the Vercel project — every deploy invalidates the entire ISR cache.

Symptom: `iucn_sync` Celery task creates `pending_review` rows for several species

Cause: those species have a `manual_expert` ConservationAssessment that disagrees with the incoming IUCN category, and the conflict hasn't been acknowledged. By design, the sync lands the IUCN row as `pending_review` and creates a `ConservationStatusConflict` rather than silently overwriting human review.

Fix: open **Species → Species → click the species → Conservation status audit** panel (Tier 3+). Review both rows, either approve the incoming IUCN row (which clears the conflict and updates the mirror) or keep the manual override and acknowledge the conflict via the admin's conflict UI.

Symptom: `seed_localities` errors with `species not found: 'X'`

Cause: the locality CSV references a species not in the registry yet.

Fix: run `seed_species --csv ...` first, or add the missing species to the species CSV. `seed_all` handles the order automatically.

Symptom: `seed_localities` flags rows as `needs_review`

Cause: coordinates fall outside Madagascar's bounding box (lat [-26.0, -11.5], lon [43.0, 51.0]) or longitude > 50.6° (offshore). Those records load successfully but are hidden from the public map until a human verifies the coordinates.

Fix: open **Species → Species localities**, filter by `needs_review=True`, fix the coordinates or confirm the row is correct and uncheck `needs_review`.

Symptom: husbandry record won't publish

Cause: the publish-time validation in `husbandry/admin.py:save_related` silently flips `published=False` if any of the three preconditions fail: ≥1 `HusbandrySource` inline, `last_reviewed_by` set, `last_reviewed_at` set.

Fix: the form should show three errors at the top. Add the source, set the reviewer + date, save again.

Symptom: GitHub deploy workflow ran but nothing changed on staging

Cause: the `workflow_run` gate skipped silently because CI was red. The deploy workflow runs but the `if:` clause short-circuits to a no-op.

Fix: check the CI workflow on the same commit. Fix CI, push the fix, let CI go green; the next push to `main` will redeploy.

---

14. Glossary

What this is for

A flat list of acronyms and domain terms used across the platform. Skim once; come back when something doesn't ring a bell.

Term	Meaning
IUCN	International Union for Conservation of Nature. Maintains the Red List of Threatened Species and the canonical category codes.
IUCN status	One of `EX, EW, CR, EN, VU, NT, LC, DD, NE`. CR/EN/VU are "threatened" and trigger coordinate generalization on the public map.
CARES	Conservation, Awareness, Recognition, and Encouragement of Species — a hobbyist breeder priority list for fishes deemed at risk in the wild. CARES species carry a `cares_status` (`CCR/CEN/CVU/CLC`).
SHOAL	A freshwater-fish-focused conservation organization. Maintains the "1,000 Fishes Blueprint" — a global priority alignment that the platform tracks via `Species.shoal_priority` and the `shoal_priority_report` worklist.
Darwin Core	A biodiversity data standard (Tyler Reed et al.) for sharing occurrence records. The platform's `SpeciesLocality` model is shaped to be Darwin-Core-exportable.
GBIF	Global Biodiversity Information Facility — the canonical aggregator of Darwin-Core occurrence data. The platform publishes outbound to GBIF via Darwin Core Archives + IPT.
ZIMS / Species360	The zoo industry's animal-records system. Institutions track captive populations there; we mirror snapshots into `ExSituPopulation` via institutional data sharing rather than direct API integration.
Ex-situ	"Out of place" — captive populations (zoos, aquariums, hobbyist programs).

In-situ	"In place" — wild populations and the field programs that work on them.
Studbook	The pedigree register for a managed captive population. In French: **livre généalogique** (the locked-term standard, *not* "registre généalogique" — see i18n-corrections-workflow.md).
EEP	EAZA Ex-situ Programme. Walks under `CoordinatedProgram(program_type='eep')`. See EAZA_EEP_ENTRY_GUIDE.md.
SSP	AZA Species Survival Plan. Walks under `CoordinatedProgram(program_type='ssp')`.
Modeltranslation	The `django-modeltranslation` package. Adds locale-suffixed columns (`description_en`, `description_fr`, ...) to a model with one line in `<app>/translation.py`.
mt_draft	First state in the i18n review pipeline. Output of `translate_species`. Machine-translated, unreviewed.
writer_reviewed	Second state. The `@conservation-writer` agent has reviewed the translation for voice/idiom; awaiting human approval.
human_approved	Third state. A reviewer (you) has signed off. With `I18N_ENFORCE_REVIEW_GATE=true`, this is the gate to public-site visibility in the target locale.
published	Fourth state. Reserved for content explicitly flagged as final / archival; treated equivalently to `human_approved` by the public-site review gate.
MFFCP	Madagascar Freshwater Fish Conservation Platform — this project's internal abbreviation, also the name of the local Postgres database (`mffcp`).
MPTT	Modified Preorder Tree Traversal — the algorithm used by `django-mptt` to store the `Taxon` hierarchy.
PostGIS	The spatial extension to PostgreSQL. Stores point geometries on `SpeciesLocality` and polygon geometries on `Watershed` and `ProtectedArea`.
HydroBASINS	A global watershed-boundaries dataset. Loaded via `load_reference_layers --watersheds`.
WDPA	World Database on Protected Areas. Loaded via `load_reference_layers --protected-areas`.
Pfafstetter code	A hierarchical hydrological coding scheme used to identify nested watersheds. Stored on `Watershed.pfafstetter_code`.
AuditEntry / AuditLog	Two separate audit logs. `audit.AuditEntry` is the structured per-field change log used by `iucn_sync` and the conservation-status governance flow. `accounts.AuditLog` is the older general-purpose one.
Caddy	The reverse-proxy / TLS terminator on the staging VM. Lives at `deploy/staging/Caddyfile`.
ISR	Incremental Static Regeneration — Next.js's caching strategy for the public pages. Default 3600s window; the manual revalidate webhook in §7 is the override path.

DRF	Django REST Framework. The API at `/api/v1/*` is built on it.
NextAuth	The auth library on the frontend. Wraps the Django `/api/v1/auth/*` endpoints with a Credentials provider; stores a JWT cookie. Server-only DRF token retrieval via `getServerDrfToken()`.
Tier 1 / 2 / 3 / 4 / 5	The five access tiers in §1. Stored as `accounts.User.access_tier` (integer).
`@conservation-writer`	A specialized Claude agent that voice-reviews translations and platform copy. Lives in `.claude/agents/conservation-writer.md`. Locked terms (e.g. studbook → livre généalogique) are codified in that file's glossary table.

Last updated: 2026-04-30. When you make a substantive operational change (new management command, new flag, new admin action), update this manual in the same PR.